Recording Train Service Status using Auckland Transport's API – Azure Function App

Clifford Dela Cruz

4/2/26

Document Version Control

Version	Date	Revised By	Summary of Changes
1.0	02/04/2026	Clifford	
1.1	08/04/2026	Clifford	Added version control, added table of contents, updated flowchart for 'Partially Running'

Contents

Objective

The objective of this project is to capture and analyse instances where Auckland's train services—specifically the Eastern, Southern, Onehunga, and Western lines—stop operating due to unplanned faults or scheduled maintenance. As a regular commuter, I aim to build a dataset that highlights the frequency and timing of these disruptions, with particular focus on those occurring during peak office hours. The goal is not to criticise Auckland Transport, but to understand underlying patterns and gain clearer visibility into how often service interruptions occur.

This phase of the project extends the original application, which previously ran on a local machine and stored data in a local database. The enhanced version now leverages a MySQL server hosted in Azure, with data collection automated through an Azure Function App written in Python. This cloud-based architecture enables reliable, scheduled execution and centralised storage, supporting long-term analysis and improved scalability.

Challenges encountered

Initially, I explored the possibility of extracting disruption information through web scraping from the Auckland Transport train line status page. However, this approach was not feasible. The website relies on internal API endpoints that are not publicly accessible, meaning the data displayed on the site cannot be retrieved programmatically.

The Auckland Transport public API also presents limitations. It does not provide a direct feed of line-level disruptions or maintenance shutdowns. Instead, the real-time API only reports information at the *trip* level—specifically, whether a trip is running or cancelled. To work around this constraint, I ingest the full real-time feed and determine whether a train line has active trips. If a line has no active trips, it is classified as "stopped."

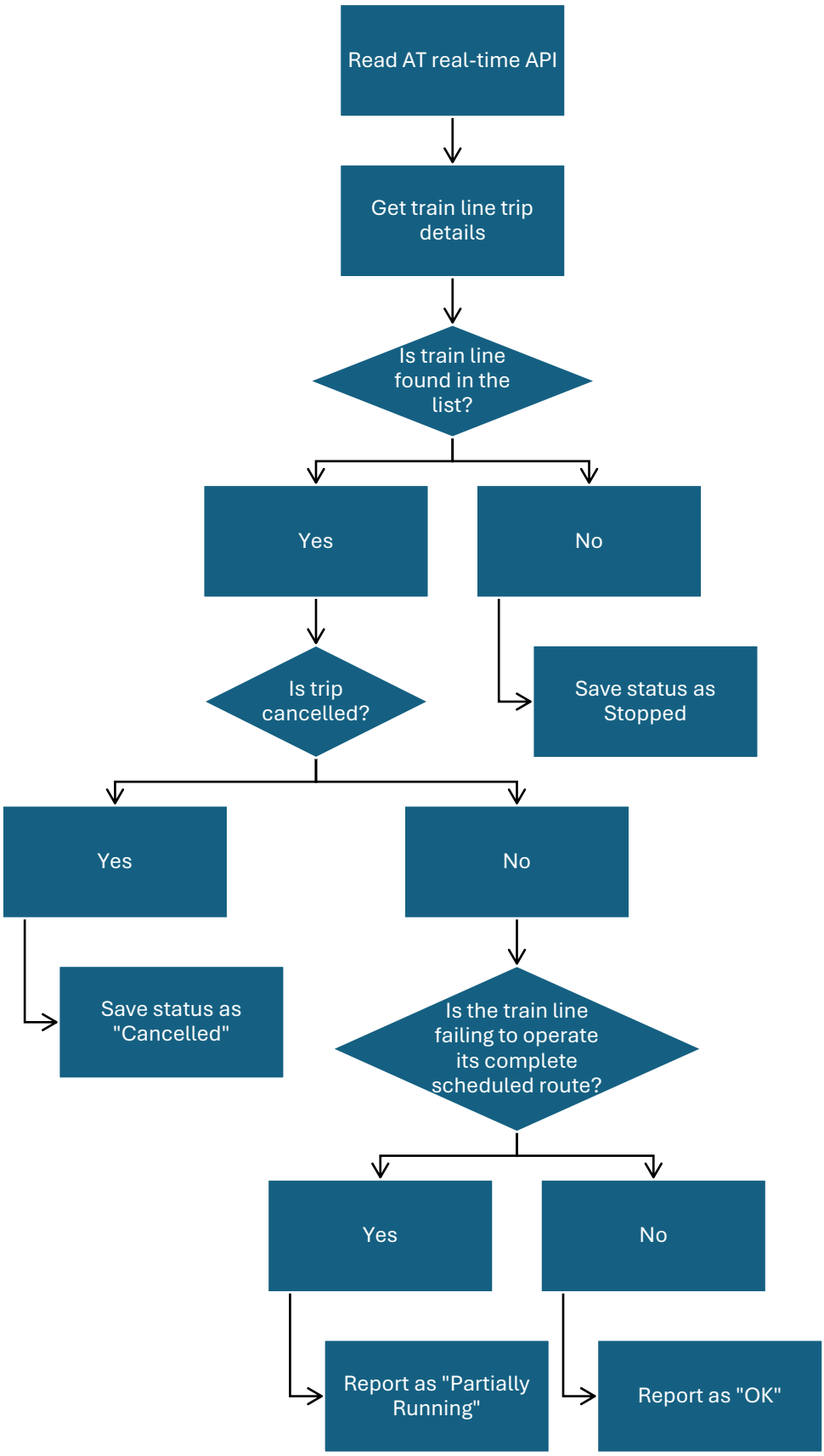
To detect partial closures (for example, when only a segment such as Puhinui–Pukekohe is closed for maintenance), each trip is evaluated based on its start and end points. By checking the trip headsigns, I can determine whether the full route is operating or whether only part of the line is active.

After completing the enhancements and successfully running the program on a local machine, the next challenge was to transition the solution into a cloud-based architecture. This required working through three key tasks:

1. **Hosting a MySQL database in Azure and establishing a secure connection to it.**
2. **Modifying the existing Python application so it could run as an Azure Function**, including adapting the structure, dependencies, and environment variable handling.
3. **Deploying the function and configuring the necessary settings** to ensure the Azure Function App could authenticate with and write data to the Azure-hosted MySQL database.

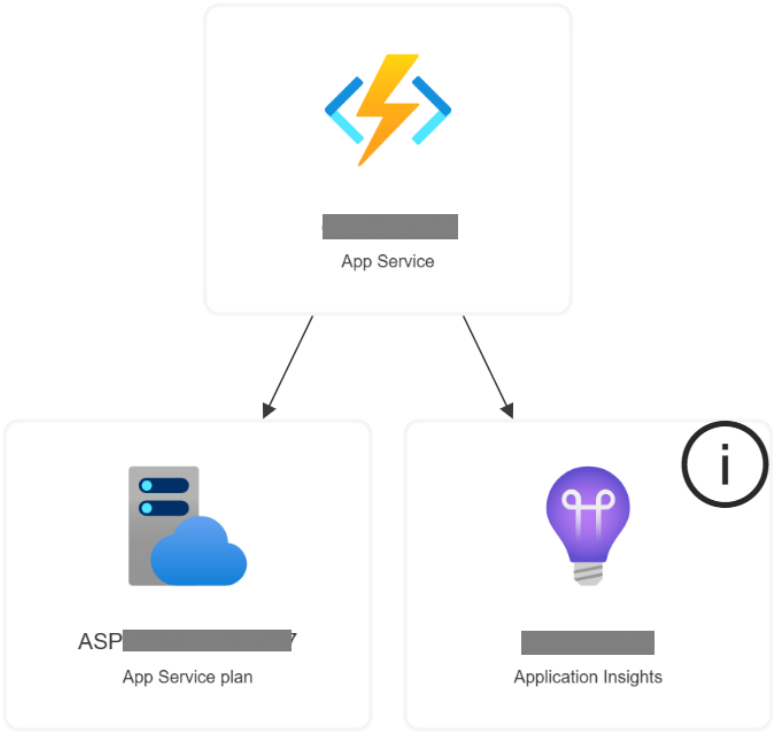
Flowchart

This is a high-level description of the program flow



Resource Visualizer (Azure)

Azure Function App



Azure Database for MySQL

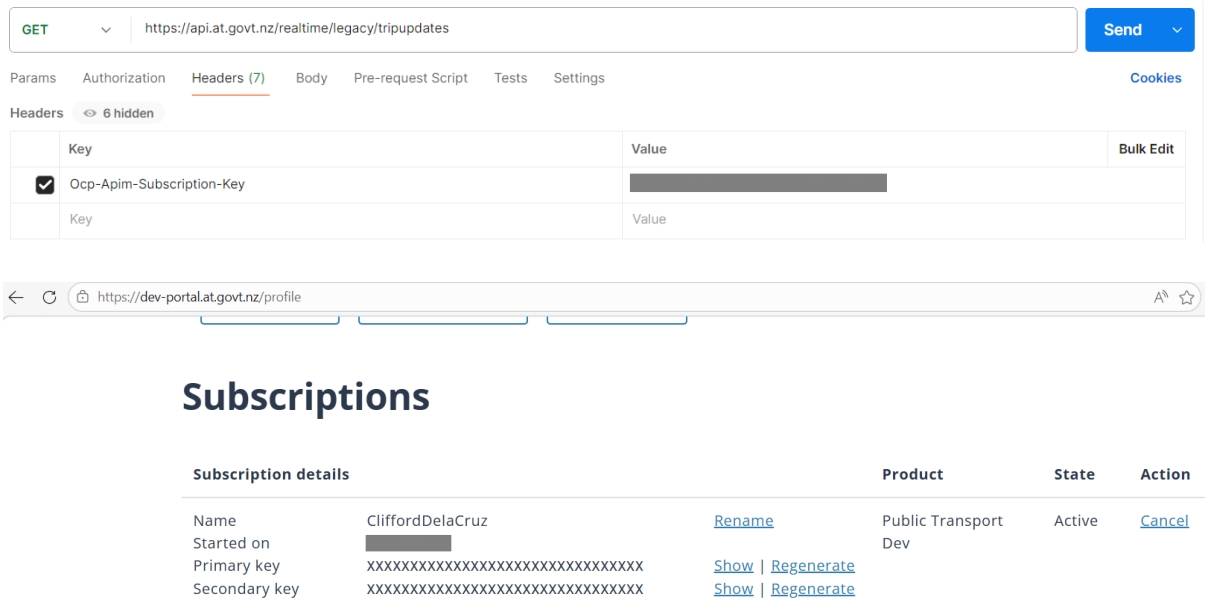
APIs used

The APIs I used are the following:

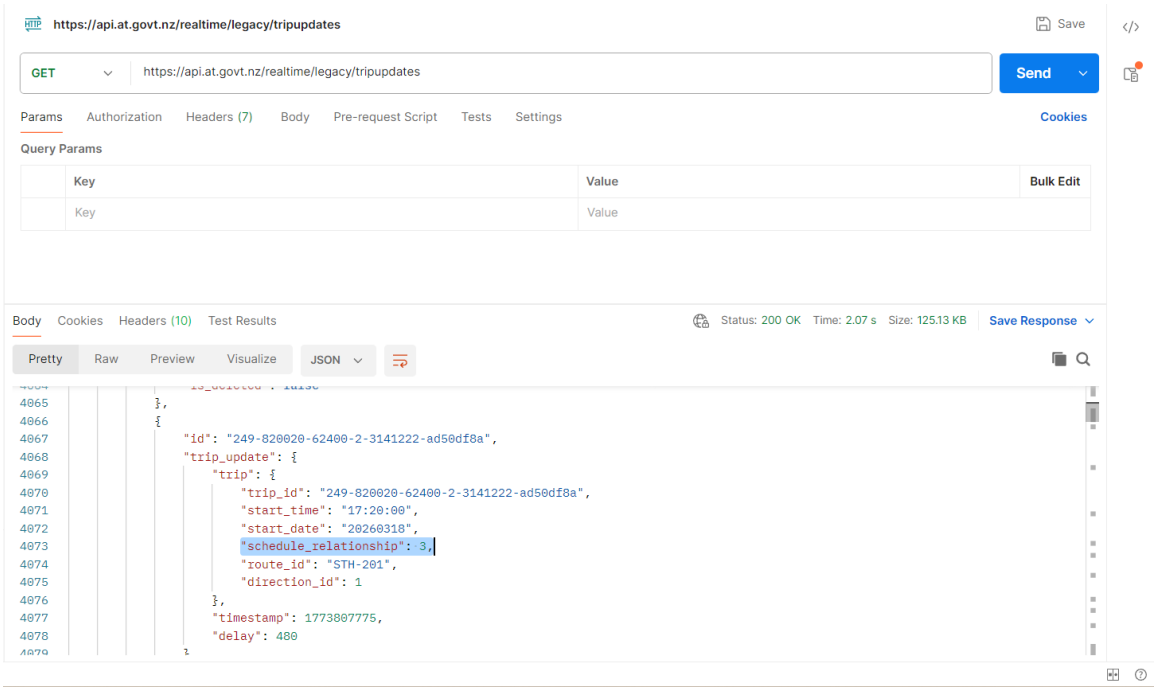
1. Real Time compat – Trip Updates: this is used to take in all the train service updates. This will provide the information such as Trip ID, the Route ID, the status of the trip.

   For this project, we are focusing on route ID and trip ID, as those contains the train line information as well as the trip information respectively.
   URL: https://api.at.govt.nz/realtime/legacy/tripupdates[?callback][&tripid][&vehicleid]
   Resource: https://dev-portal.at.govt.nz/api-details#api=gtfs-realtime-compat&operation=get_trip_updates

   Prerequisites – need an API key (Ocp-Apim-Subscription-Key) which can be acquired by registering in https://dev-portal.at.govt.nz/



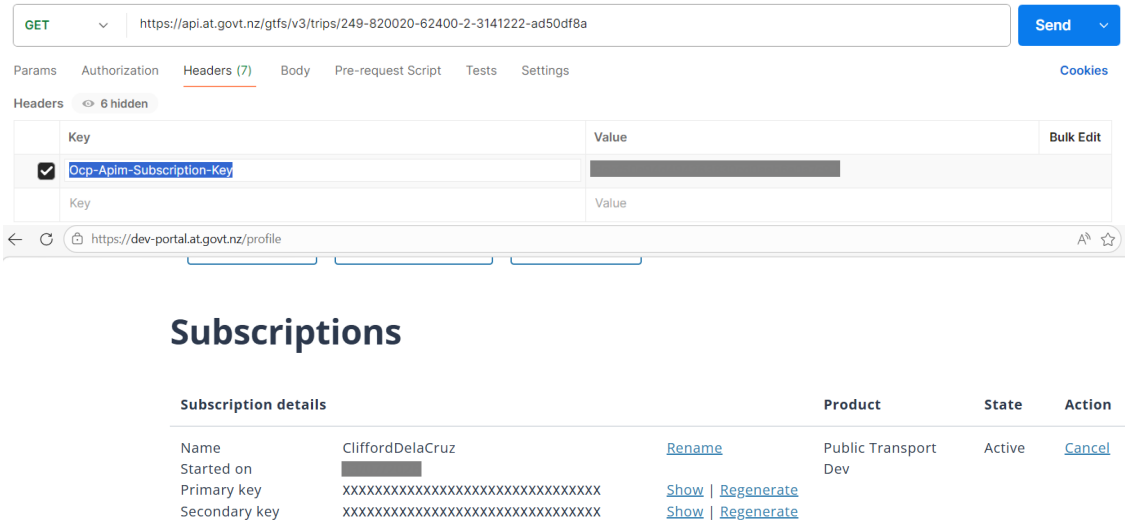
Sample images of key header input in Postman and subscription information



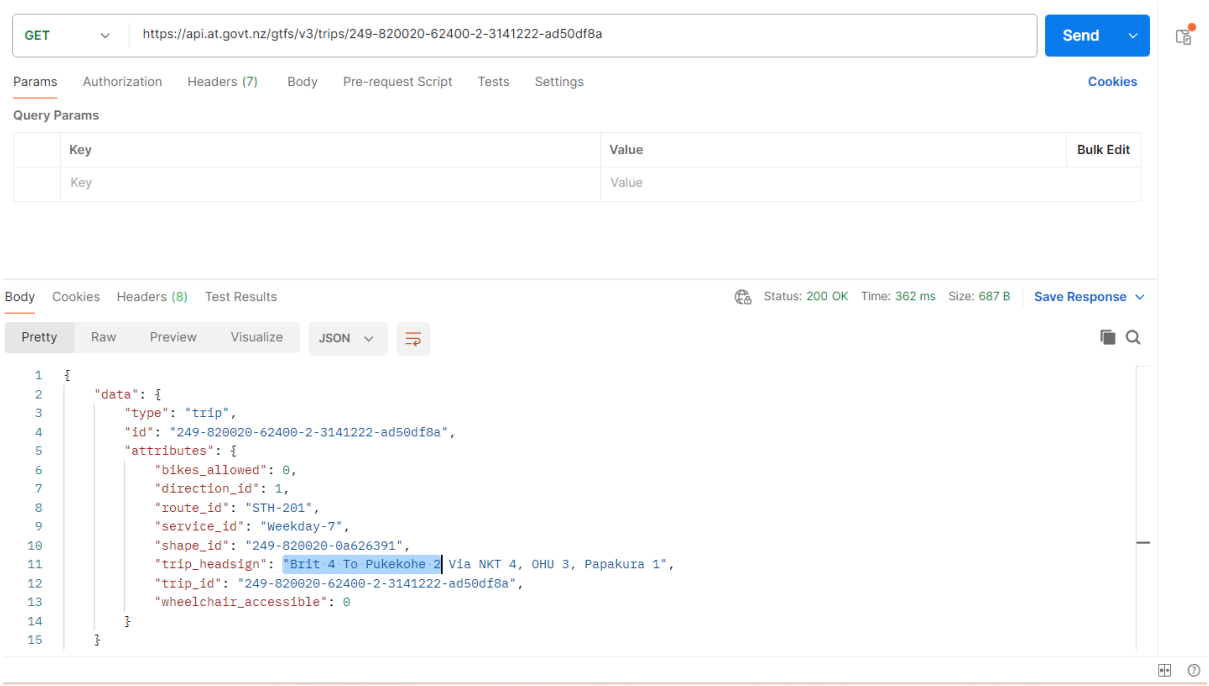
Example 1.1 This image shows one result of a trip in Southern line that is cancelled.
(schedule relationship = 3 means trip is cancelled)

2. General Transit Feed V3 API – Trip by ID: this is used to identify the trip ID to indicate the services covered in that trip. Example, trip id 246-850001-30660-2-4234101-0620e5fd is an Eastern line service trip, which is a trip from Manukau to Britomart
URL: https://api.at.govt.nz/gtfs/v3/trips/{id}
Resource: https://dev-portal.at.govt.nz/api-details#api=gtfs-api&operation=get-trips-id

   Prerequisites – need an API key (Ocp-Apim-Subscription-Key) which can be acquired by registering in https://dev-portal.at.govt.nz/



Sample images of key header input in Postman and subscription information



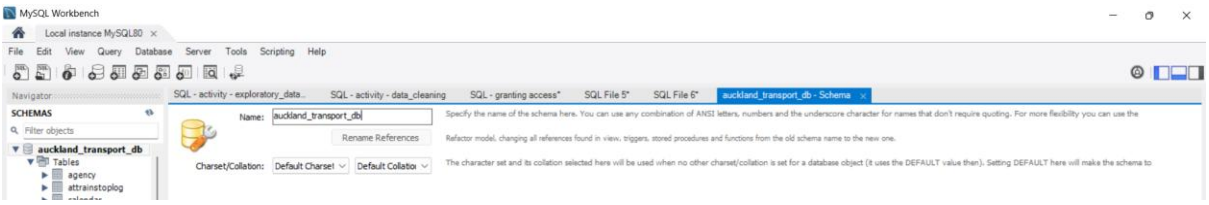
Example 2.1 This image shows that the trip information that it is a train service from Britomart to Pukekohe

Additional links for reference (API)

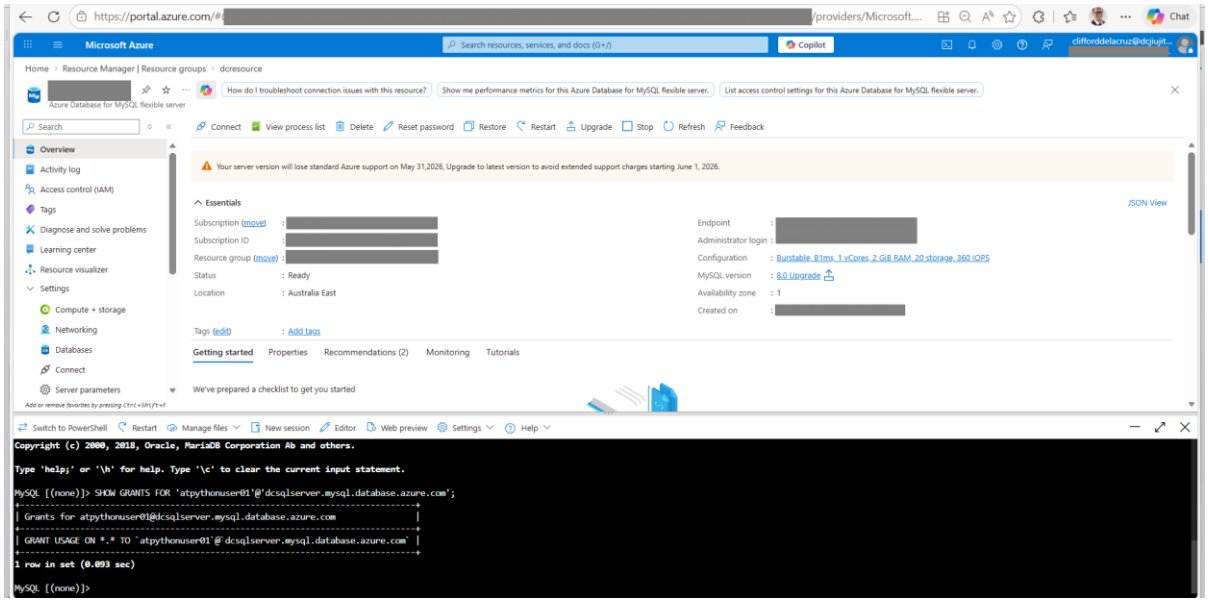
https://dev-portal.at.govt.nz/realtime-api
https://gtfs.org/documentation/schedule/reference/

Database setup

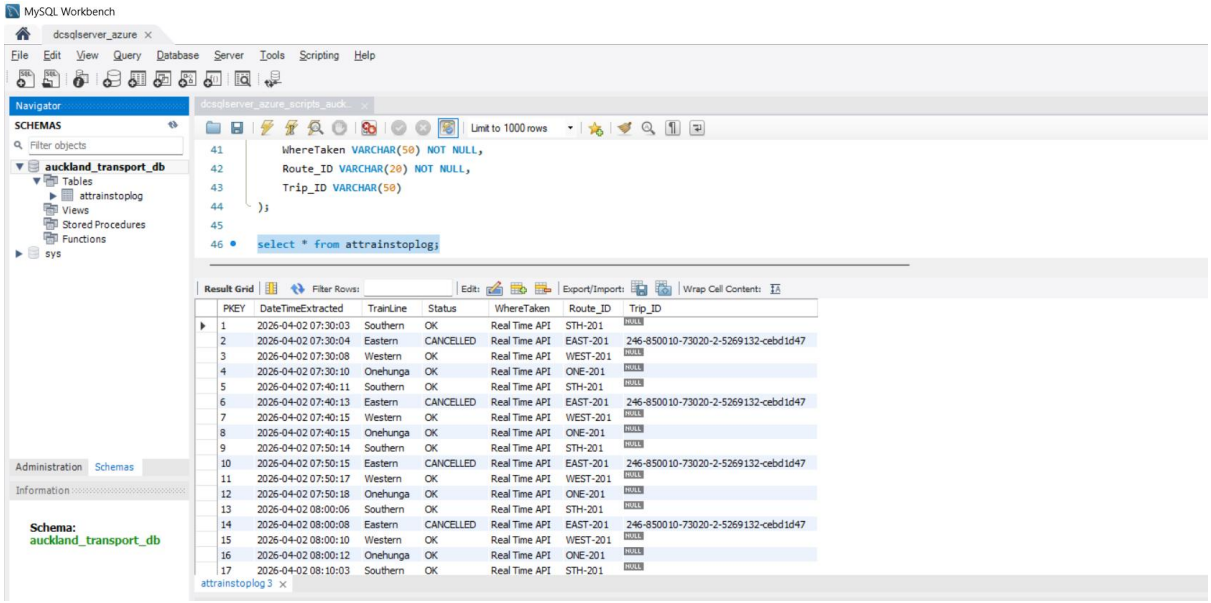
I used MySQL and created a schema "auckland_transport_db"



I created an account that can access the database and granted CREATE, DROP, SELECT, INSERT, UPDATE, DELETE role. This role will then be used in the program to be able to update a table "attrainstoplog".



Running the "attrainstoplog" will show the information on the strain line during the program call

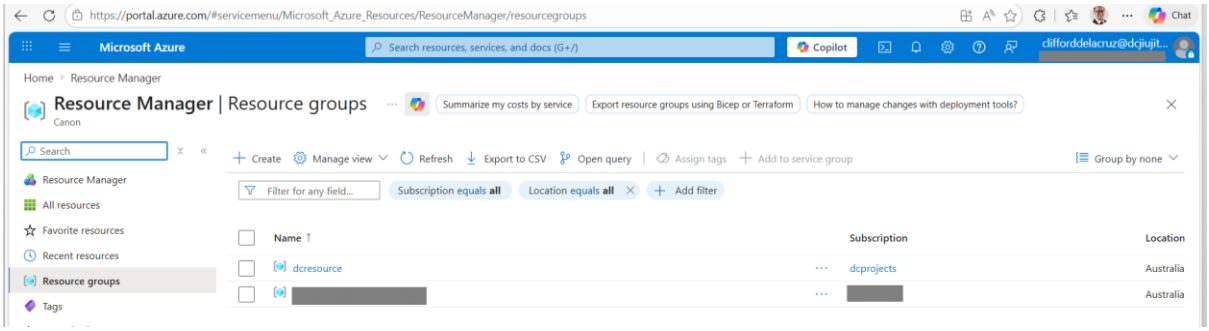


Setup and configuration steps

1. Go to Azure and setup resource group and corresponding resource

1.1 Create a Resource Group

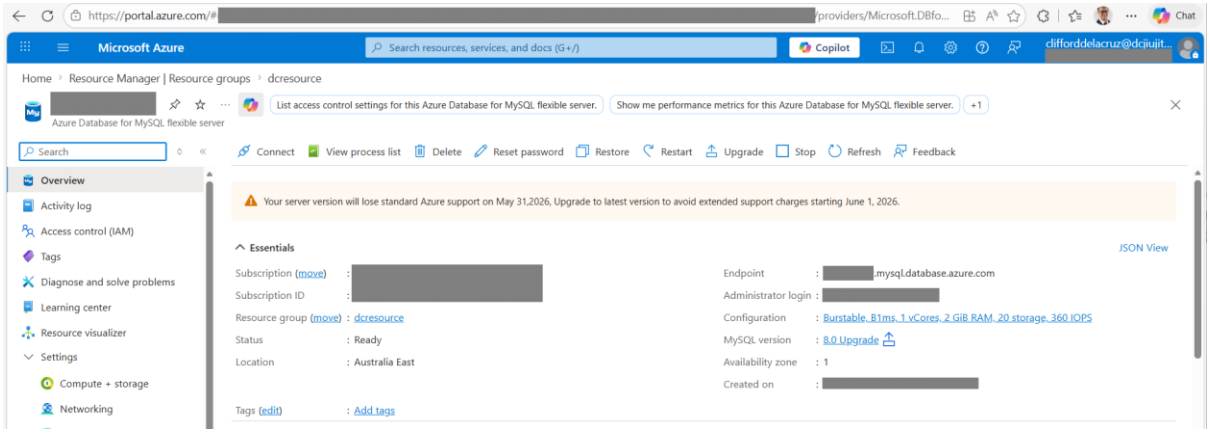
- Name: *[resource_name]* – in this project I used 'dcresource' as a resource group
- Region: **Australia East**

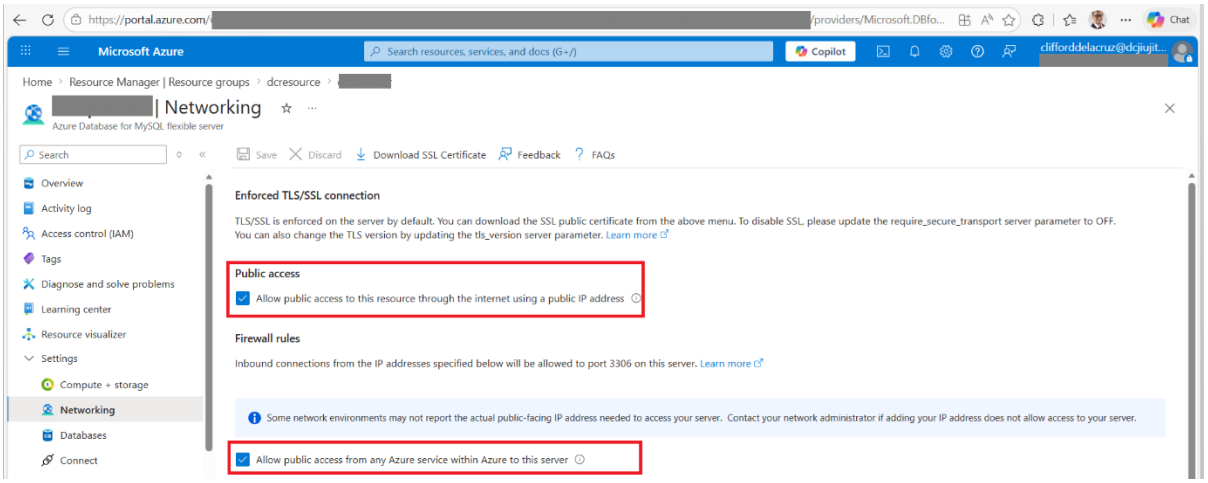


1.2 Create Azure MySQL Flexible Server (Cheapest Settings)

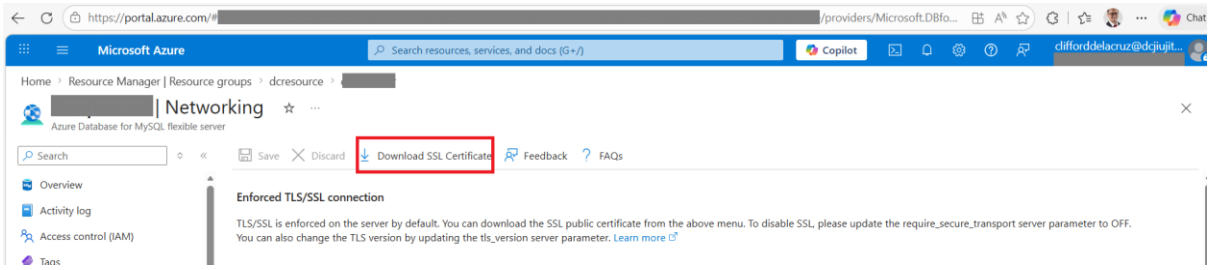
Settings:

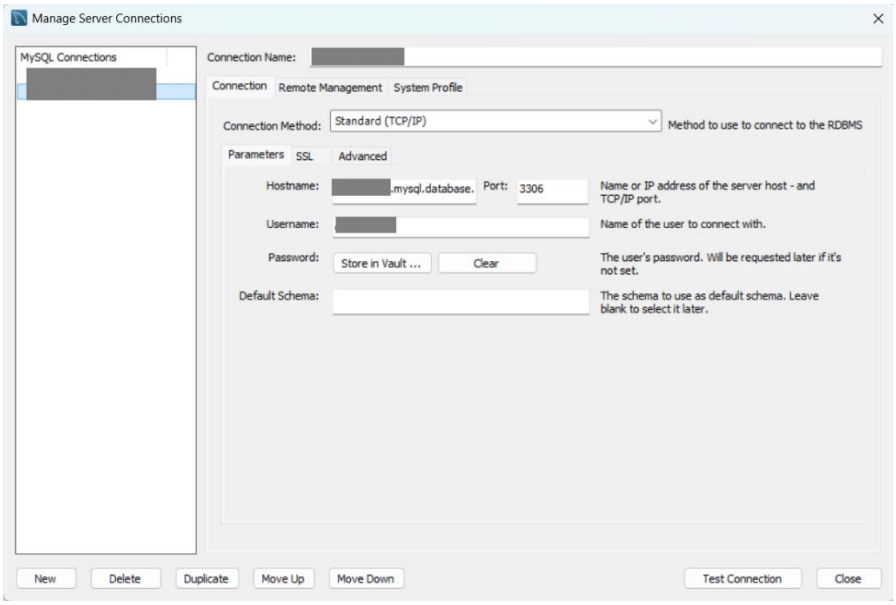
- Compute: **Burstable B1ms**
- Storage: **20GB (minimum)**
- Backup retention: **7 days**
- Server name: *[your_sqlserver_name]*
- Networking:
  - Public access
  - "Allow public access from Azure services"

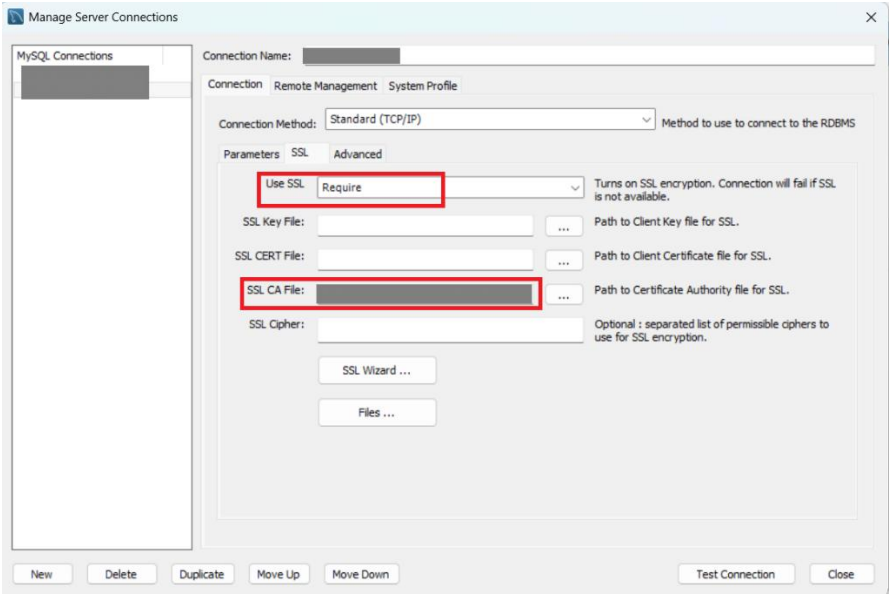




The database can also be accessed using MySQL Workbench. Note that you need to download the SSL certificate







Create database:

```
CREATE DATABASE auckland_transport_db;
```

Create user account that will be used to insert data (

```
CREATE USER '[userid]'@'%' IDENTIFIED BY '[Password]';
GRANT ALL PRIVILEGES ON auckland_transport_db.* TO '[userid]'@'%';
FLUSH PRIVILEGES;
```

Create the table attrainstoplog

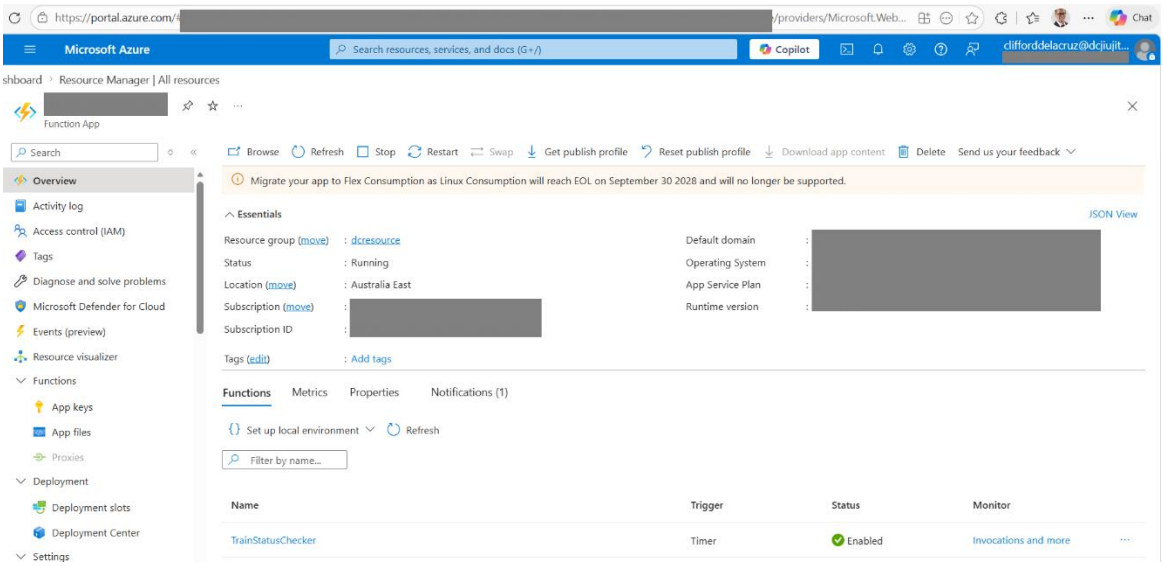
```
CREATE TABLE attrainstoplog (
  PKEY INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  DateTimeExtracted DATETIME NOT NULL,
  TrainLine VARCHAR(50) NOT NULL,
  Status VARCHAR(30) NOT NULL,
  WhereTaken VARCHAR(50) NOT NULL,
  Route_ID VARCHAR(20) NOT NULL,
  Trip_ID VARCHAR(50)
);
```

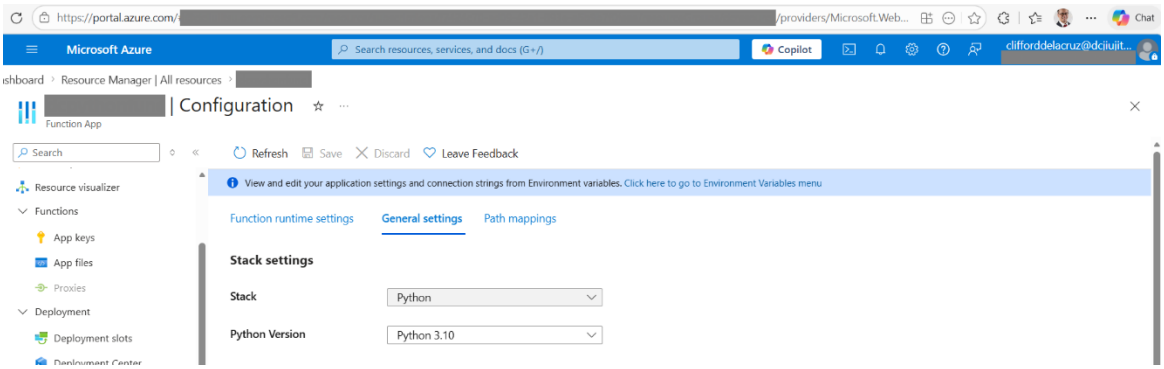
2. Create an Azure Function App resource and modify the python script

2.1 Create Azure Function App (Python)

Settings:

- Publish: **Code**
- Runtime: **Python 3.10**
- Hosting: **Consumption Plan**
- Region: **Australia East**
- Storage: auto-create



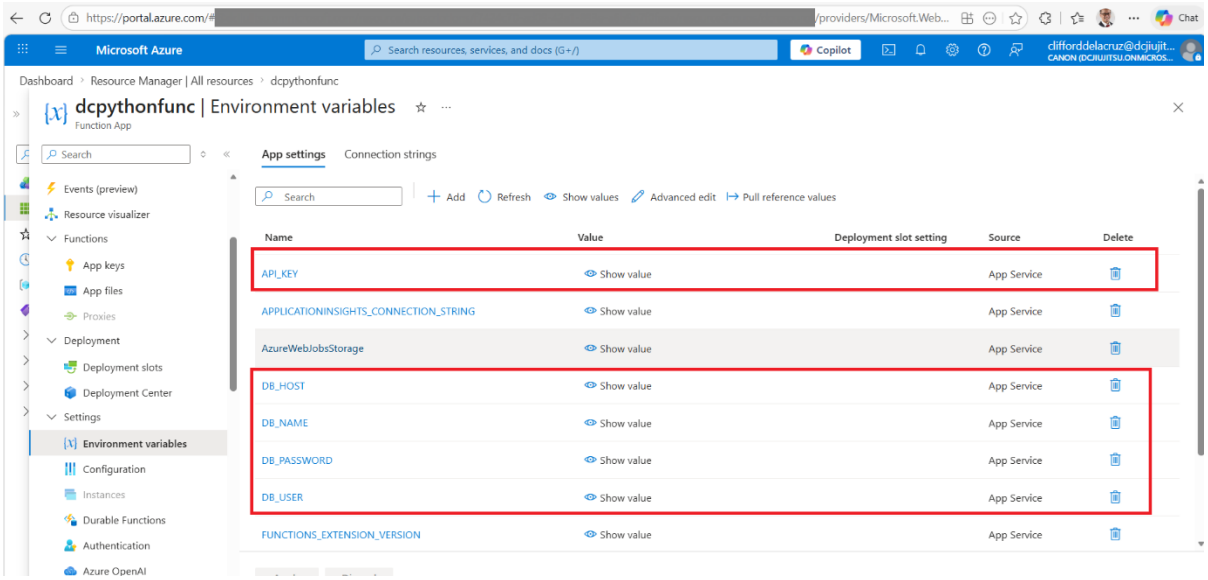


Step 2.2 — Add App Settings (Environment Variables)

In your Function App:

Settings ➔ Environment variables ➔ App Settings:

Key	Value
DB_HOST	*[your_sqlserver_name]*.mysql.database.azure.com
DB_USER	*[userid]*
DB_PASSWORD	*[Password]*
DB_NAME	auckland_transport_db
API_KEY	*[your AT API key]*



Step 2.3 Modifications on the python script

Replace the main block to run with functions and add azure.functions library. Note that the Azure function does not run in __main__

```
import azure.functions as func

def main(mytimer: func.TimerRequest):
    logging.info("Timer trigger function started")
    entities = fetch_tripupdates()
    status = classify_lines(entities)
```

Replace the hardcoded setting into the environmental variable earlier set in Azure

```
userid = os.environ["DB_USER"]
password = os.environ["DB_PASSWORD"]
host = os.environ["DB_HOST"]
database = os.environ["DB_NAME"]
port = 3306
```

Important: The python script must be named **__init__.py**

Step 3 — Create Timer Trigger

Name the file **function.json**:

```
{
  "scriptFile": "__init__.py",
  "bindings": [
    {
      "name": "mytimer",
      "type": "timerTrigger",
      "direction": "in",
      "schedule": "0 */10 * * * *"
    }
  ]
```

```
}
```

This runs every 10 minutes.

Step 4 — Create host.json
This is a requirement
This is the content:

```
{
  "version": "2.0"
}
```

Step 5 – Create requirements.txt
This is a requirement if there are packages to be installed
This is the content:

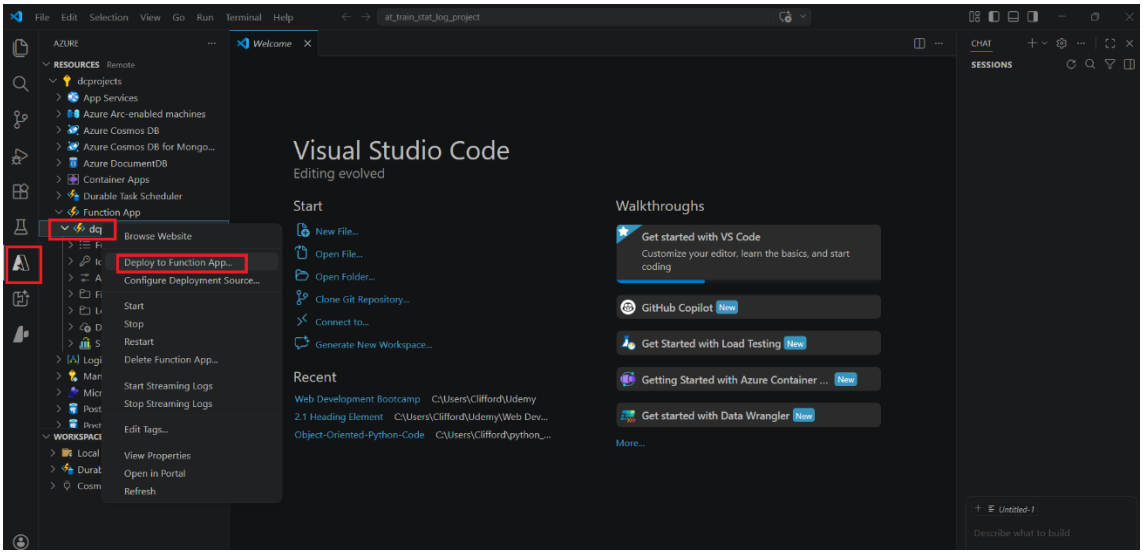
```
azure-functions
requests
sqlalchemy
mysql-connector-python
```

Step 6 – Deploy from VS Code
Note: Make sure that the folder structure is correct and loaded as a workspace.

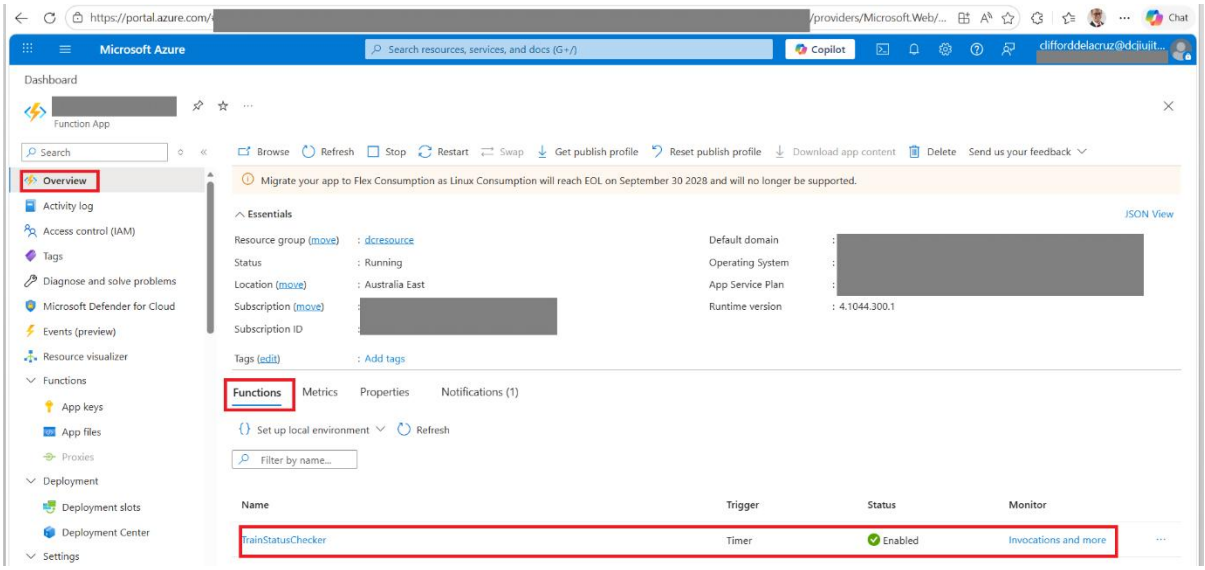
Root_Folder /
host.json
requirements.txt
Function_Folder /
__init__.py
function.json

1. Install:
   - Azure Functions extension
   - Azure CLI
   - Azure Functions Core Tools
2. Open your folder and look for the function app
3. Right-click **Function App → Deploy**
4. Confirm overwrite

Important: make sure that the python interpreter is 3.10. As of this writing, the python version above 3.10 will not successfully deploy the function. To make sure of this, install python 3.10 and set it as the interpreter in VS code by typing CTRL+SHIFT+P and type "***python: select interpreter***"



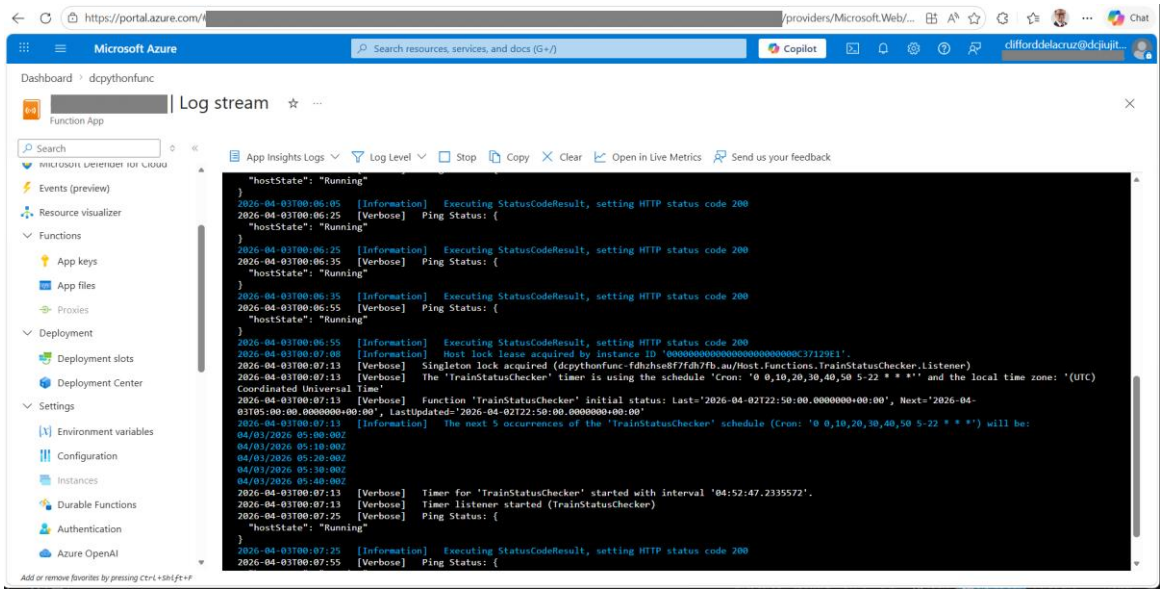
To confirm if the function is imported, it should be listed on the **Functions tab** in **Overview section**



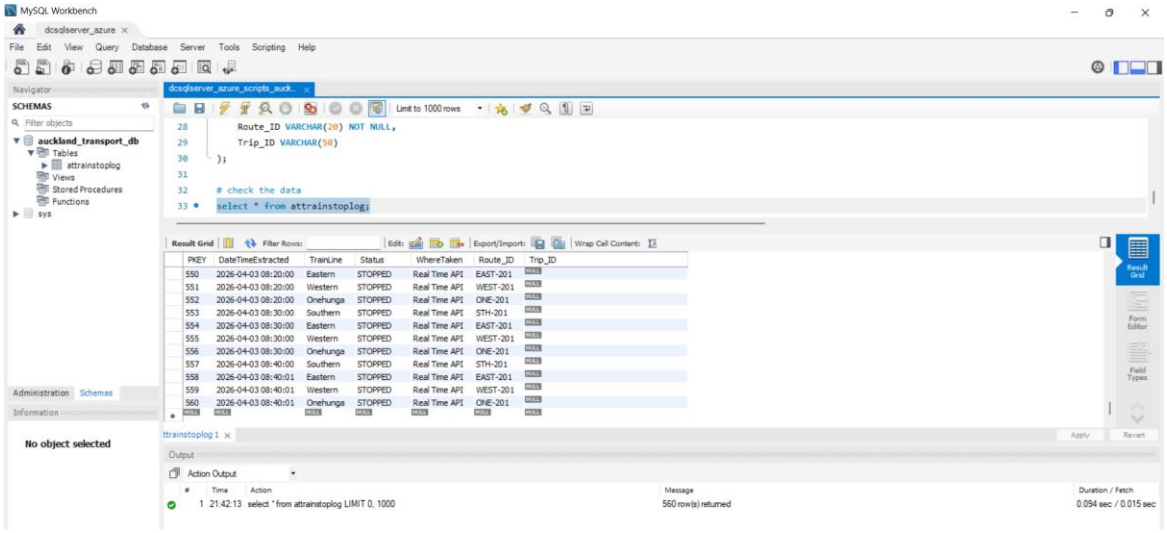
Step 7 — Test

In Azure Portal:

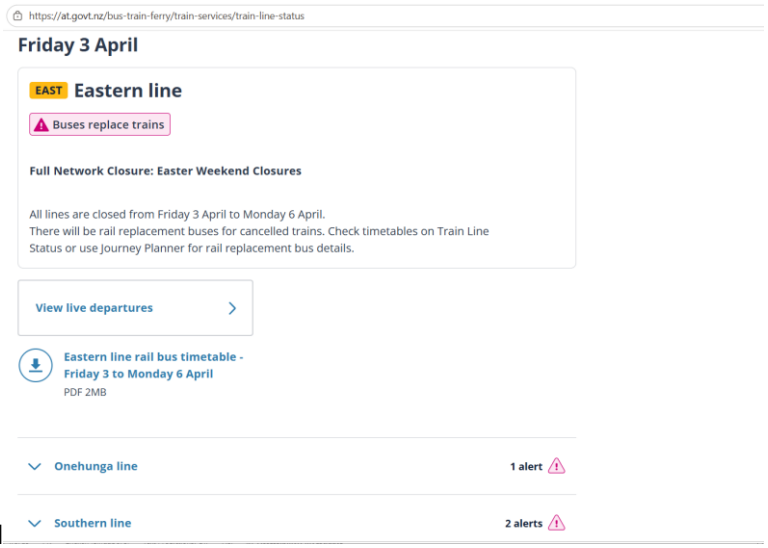
- Go to Function App ➔ **Monitor**
- Check logs
- Confirm MySQL rows are being inserted



Data populated in attrainstoplog table. Note that the date captured is in **UTC**



To show that the code works, here is the status in Trains running today in Auckland on 3 April 2026:



N